EduAgentic
AGENTIC AI FOR EDUCATION
MCP · A2A · Jours 33 & 34 · v1.0

JOUR 34 · MINI-PROJET · CORRECTION FINALE

A2A en EdTech : faire collaborer des agents spécialisés

AgentCards, découverte, délégation JSON-RPC et correlation_id

AUTEUR	Kamal El Afoui
PROGRAMME	Centre des Très Petites Entreprises Solidaires · DBA Technology
DATE	11 novembre 2026
VERSION	1.0.0 — Correction finale

CE QUE COUVRE CE DOCUMENT

- Trois agents : Tuteur, Évaluation et Orientation.
- Un orchestrateur qui découvre les AgentCards et délègue en parallèle.
- Un protocole JSON-RPC 2.0 avec message/send et tasks/get.
- Un correlation_id unique pour relier chaque échange.
- Workflow Langflow, dashboard 8011 et vidéo de démonstration.

Données 100 % synthétiques · aucun apprenant réel · dépôt reproductible

Résumé exécutif

Un seul agent ne possède pas toutes les compétences. Le protocole Agent-to-Agent (A2A) permet à des agents spécialisés de se découvrir, de se déléguer des tâches et de collaborer de manière standardisée. Chaque agent publie une AgentCard qui décrit son identité, ses compétences, son endpoint et ses conditions d'utilisation.

Ce mini-projet construit EduA2A : un orchestrateur, un agent Tuteur (cours et parcours), un agent Évaluation (quiz et feedback) et un agent Orientation (écart de compétences et trajectoire métier). L'orchestrateur découvre les cartes, appelle les agents en parallèle via JSON-RPC et agrège leurs réponses. Chaque délégation porte un correlation_id, visible dans la timeline du dashboard et dans les logs.

Tableau 1 · Synthèse des résultats du mini-projet J34.

Indicateur	Valeur	Preuve
Agents spécialisés	3 (+ 1 orchestrateur)	agent_cards/*.json
AgentCards publiées	4	GET /.well-known/agent.json
Délégations par demande	3	timeline du dashboard
Découverte HTTP	3 / 3 agents	GET /discover
Latence d'orchestration	49.2 ms	test HTTP du 11/11/2026
Taux de succès (jeu de 84 traces)	92.9 %	data/a2a_delegations.csv
Correlation_id	unique par demande	data/a2a_live.jsonl

Résultat principal

L'orchestrateur n'a pas besoin de connaître l'implémentation des agents : il lit leurs cartes, sélectionne les compétences et délègue. La réponse finale est une agrégation traçable de trois expertises.

Sommaire

Résumé exécutif 2
1. Contexte et problématique 4
1.1 Pourquoi un protocole A2A ? 4
1.2 Les trois piliers 4
2. Objectifs et livrables 5
3. Architecture 6
3.1 Cycle d'une demande 6
4. AgentCards 7
4.1 Structure d'une carte 7
4.2 Les quatre cartes 7
5. Protocole JSON-RPC 8
5.1 Méthodes 8
5.2 Exemple de requête 8
5.3 Exemple de réponse 8
6. Orchestration et sélection 9
6.1 Algorithme 9
6.2 Répartition des rôles 9
7. Workflow Langflow 10
7.2 Workflow Discovery seul 10
8. Démonstration et traces 11
8.1 Exemple de trace 11
8.2 Ce que le correlation_id permet 11
8.3 Bénéfices pour l'architecture multi-agents 11
9. Résultats et validation 12
10. Sécurité et limites 13
11. Mode d'emploi 14
Conclusion 15
Annexes 16
A. Glossaire 16
B. Checklist de recette 16

1. Contexte et problématique

1.1 Pourquoi un protocole A2A ?

Dans une plateforme EdTech, les besoins sont hétérogènes : rechercher des ressources, évaluer des compétences, orienter vers un métier, produire un feedback. Un agent généraliste devient vite complexe, coûteux et difficile à évaluer. La spécialisation améliore la qualité, mais crée un nouveau problème : comment faire dialoguer ces agents ?

- Chaque agent annonce ses capacités et son endpoint.
- Les autres agents peuvent le découvrir sans intégration codée en dur.
- Une tâche est confiée au spécialiste le plus adapté.
- Chaque échange est tracé pour diagnostiquer les erreurs et mesurer les performances.

1.2 Les trois piliers

Tableau 2 · Les trois piliers du protocole A2A.

Pilier	Question	Dans EduA2A
AgentCard	Qui est l'agent et que sait-il faire ?	Carte JSON sur /.well-known/agent.json
Discovery	Quels agents sont disponibles ?	GET des cartes des ports 8101-8103
Delegation	Qui fait quoi, et comment suivre ?	message/send JSON-RPC + correlation_id

2. Objectifs et livrables

- Publier une AgentCard par agent, avec identité, compétences, endpoint et conditions d'utilisation.
- Implémenter un orchestrateur qui découvre les cartes et sélectionne les agents.
- Déléguer les tâches en JSON-RPC 2.0 et agréger les résultats.
- Tracer chaque échange avec un correlation_id et des logs horodatés.
- Démontrer le tout dans un dashboard et un workflow Langflow.

Tableau 3 · Livrables du jour 34.

Livrable	Fichier	Contenu
Agents A2A	a2a/agents.py, a2a/protocol.py	Tuteur, Évaluation, Orientation
Orchestrateur	a2a/orchestrator.py	Découverte, sélection, délégation, agrégation
AgentCards	a2a/agent_cards/*.json	4 cartes A2A
Workflow Langflow	json/Agentic_AI_EdTech_J34_A2A_Final.json	Découverte + orchestration
Documentation	docs/Documentation_J34_A2A_EdTech.docx / .pdf	Ce document
Vidéo	videos/J34_A2A_Workflow_Playground.mp4	Workflow + playground

3. Architecture

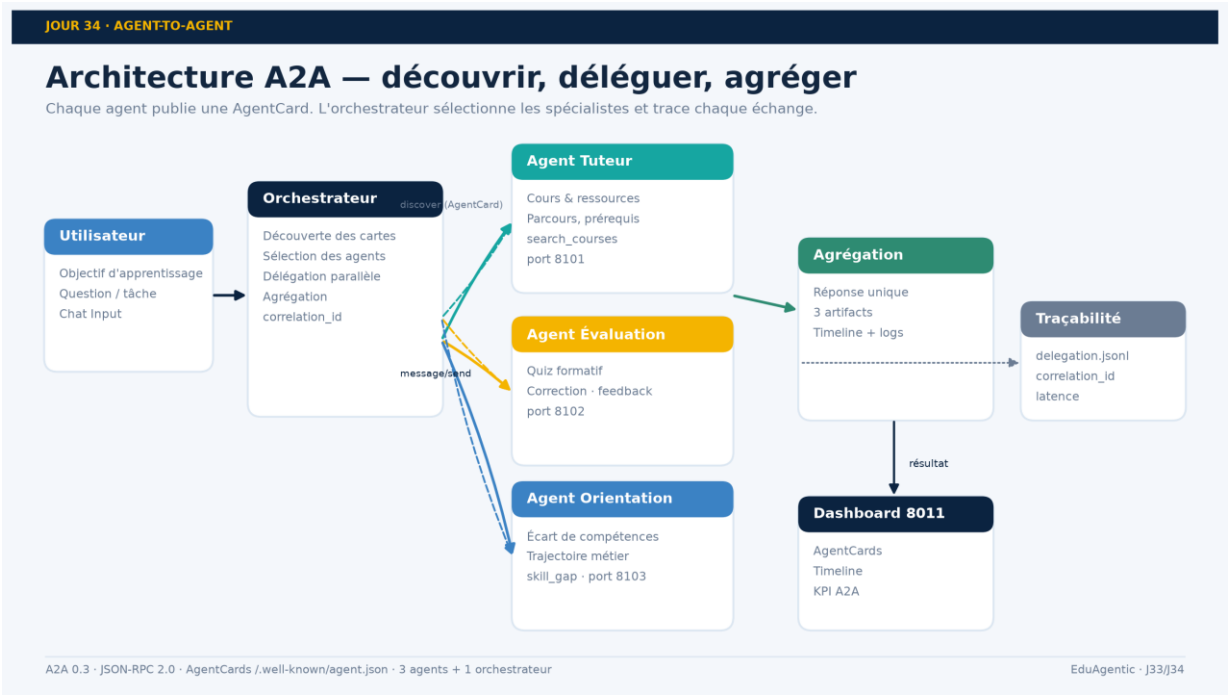


Figure 1 · Architecture A2A : découverte des AgentCards, délégation parallèle et agrégation traçable.

Tableau 4 · Composants du système A2A.

Composant	Rôle	Port	Technologie
Orchestrateur	Découvre, sélectionne, délègue, agrège	8001	FastAPI + JSON-RPC
Agent Tuteur	Cours, ressources, parcours	8101	FastAPI + AgentCard
Agent Évaluation	Quiz, correction, feedback	8102	FastAPI + AgentCard
Agent Orientation	Gap de compétences, trajectoire	8103	FastAPI + AgentCard
Dashboard	Observation, formulaire, timeline	8011	Streamlit

Les agents partagent le même backend de données que le serveur MCP : MongoDB Atlas (collections edtech_*) si MONGODB_URI est configuré, sinon les fichiers JSON/CSV. Ce choix garantit que le parcours, les ressources et le quiz portent sur les mêmes données, sans duplication.

3.1 Cycle d'une demande

- L'utilisateur formule un objectif et une question dans le dashboard.
- L'orchestrateur découvre les AgentCards disponibles et vérifie leur état.
- Il sélectionne les agents dont les compétences correspondent à la demande.
- Il envoie trois messages JSON-RPC en parallèle, avec le même correlation_id.
- Il agrège les artifacts en une réponse unique, puis écrit les traces.

4. AgentCards

4.1 Structure d'une carte

```json
{
  "name": "Agent Tuteur",
  "description": "Recherche des ressources et recommande un parcours personnalisé.",
  "url": "http://127.0.0.1:8101/",
  "version": "1.0.0",
  "protocolVersion": "0.3.0",
  "preferredTransport": "JSONRPC",
  "capabilities": {
    "streaming": false,
    "pushNotifications": false
  },
  "skills": [
    {
      "id": "recommend_learning_path",
      "name": "Parcours personnalisé",
      "tags": [
        "parcours",
        "prérequis",
        "compétences"
      ]
    }
  ]
}
```

4.2 Les quatre cartes

Tableau 5 · AgentCards publiées par EduA2A.

Agent	Compétences	Endpoint	Conditions
Orchestrateur	orchestrate_learning_request	8001	Découverte et agrégation
Tuteur	search_courses, recommend_learning_path	8101	Usage pédagogique, sources citées
Évaluation	generate_quiz, grade_quiz	8102	Feedback formatif, pas de note certificative
Orientation	skill_gap, career_path	8103	Conseil, décision humaine finale

Point d'entrée de toute collaboration

Publier une AgentCard est indispensable avant toute découverte ou délégation. Sans carte, les autres agents ne connaissent ni les compétences ni l'endpoint du spécialiste.

5. Protocole JSON-RPC

5.1 Méthodes

Tableau 6 · Méthodes A2A implémentées.

Méthode	Paramètres	Résultat
message/send	message {parts, contextId}, metadata {correlation_id}	Task avec artifacts
tasks/get	id de la tâche	État et artifacts de la tâche

5.2 Exemple de requête

```JSON
{
  "jsonrpc": "2.0",
  "id": "req-7f3a",
  "method": "message/send",
  "params": {
    "message": {
      "role": "user",
      "parts": [
        {
          "kind": "text",
          "text": "Construis mon parcours Data Analyst"
        }
      ],
      "messageId": "msg-91c2",
      "contextId": "A2A-261111-120000-AB12"
    },
    "metadata": {
      "correlation_id": "A2A-261111-120000-AB12",
      "learner_id": "L001",
      "target_skill": "data-analysis"
    }
  }
}
```

5.3 Exemple de réponse

```JSON
{
  "jsonrpc": "2.0",
  "id": "req-7f3a",
  "result": {
    "task": {
      "kind": "task",
      "id": "task-3b7c",
      "contextId": "A2A-261111-120000-AB12",
      "status": {
        "state": "completed"
      },
      "artifacts": [
        {
          "name": "result",
          "parts": [
            {
              "kind": "text",
              "text": "{...résultat JSON...}"
            }
          ]
        }
      ],
      "metadata": {
        "correlation_id": "A2A-261111-120000-AB12"
      }
    }
  }
}
```

6. Orchestration et sélection

6.1 Algorithme

- **Découverte :** lire les cartes des trois agents (HTTP, timeout court).
- **Sélection :** conserver les agents dont les compétences couvrent la demande.
- **Parallélisme :** envoyer les trois délégations avec ThreadPoolExecutor.
- **Agrégation :** fusionner profil, parcours, ressources et quiz.
- **Traçabilité :** écrire une ligne par délégation dans data/a2a_live.jsonl.

6.2 Répartition des rôles

Tableau 7 · Répartition des tâches entre agents.

Étape	Agent	Sortie
Diagnostic	Orientation	Compétences acquises, manquantes, trajectoire
Ressources	Tuteur	Cours, parcours ordonné, prérequis
Évaluation	Évaluation	Quiz de 3 questions et feedback
Synthèse	Orchestrateur	Réponse unique + timeline + correlation_id

7. Workflow Langflow

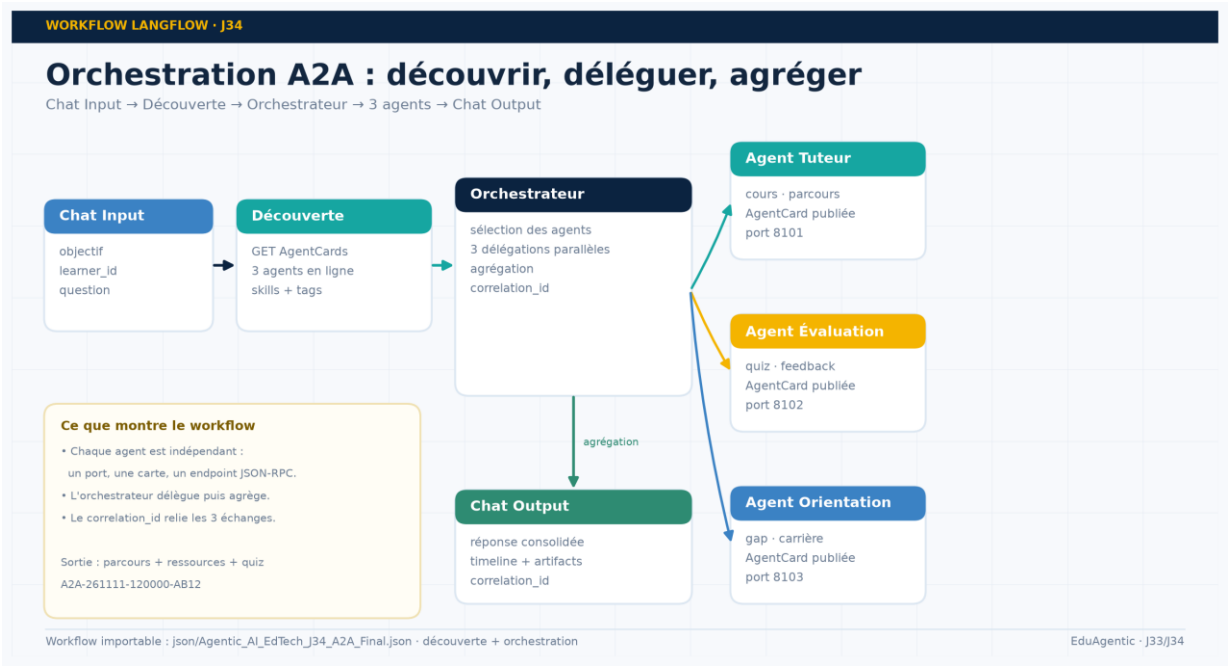


Figure 2 · Workflow A2A : découverte, orchestration, délégation aux trois agents et agrégation.

Le workflow importable se trouve dans json/Agentic_AI_EdTech_J34_A2A_Final.json. Il contient un Chat Input, un composant de découverte des AgentCards, un composant d'orchestration, un Chat Output et des notes d'architecture. L'orchestrateur appelle le service local http://127.0.0.1:8001/orchestrate.

7.2 Workflow Discovery seul

Le cours distingue deux phases : Discovery (découvrir l'agent et lire sa carte) et Delegation (envoyer la tâche à l'agent). Le workflow principal réalise les deux. Un workflow séparé, json/Agentic_AI_EdTech_J34_A2A_Discovery.json, isole la phase de découverte : Chat Input → AgentCardDiscovery → Chat Output. Il ne délègue aucune tâche ; il récupère et affiche la carte.

Tableau 8b · Phase Discovery, sans délégation.

Étape	Action	Résultat
1	Chat Input reçoit la demande	Message utilisateur
2	GET /.well-known/agent.json	JSON AgentCard
3	Lecture des champs name, description, url, skills	Données structurées
4	Chat Output affiche la carte	Identité + compétences + endpoint

```HTTP
GET http://127.0.0.1:8101/.well-known/agent.json HTTP/1.1
Accept: application/json

200 OK
{
  "name": "Agent Tuteur",
  "url": "http://127.0.0.1:8101/",
  "protocolVersion": "0.3.0",
  "skills": [
    {"id": "search_courses", "name": "Recherche de cours"},
    {"id": "recommend_learning_path", "name": "Parcours personnalisé"}
  ]
}
```

8. Démonstration et traces

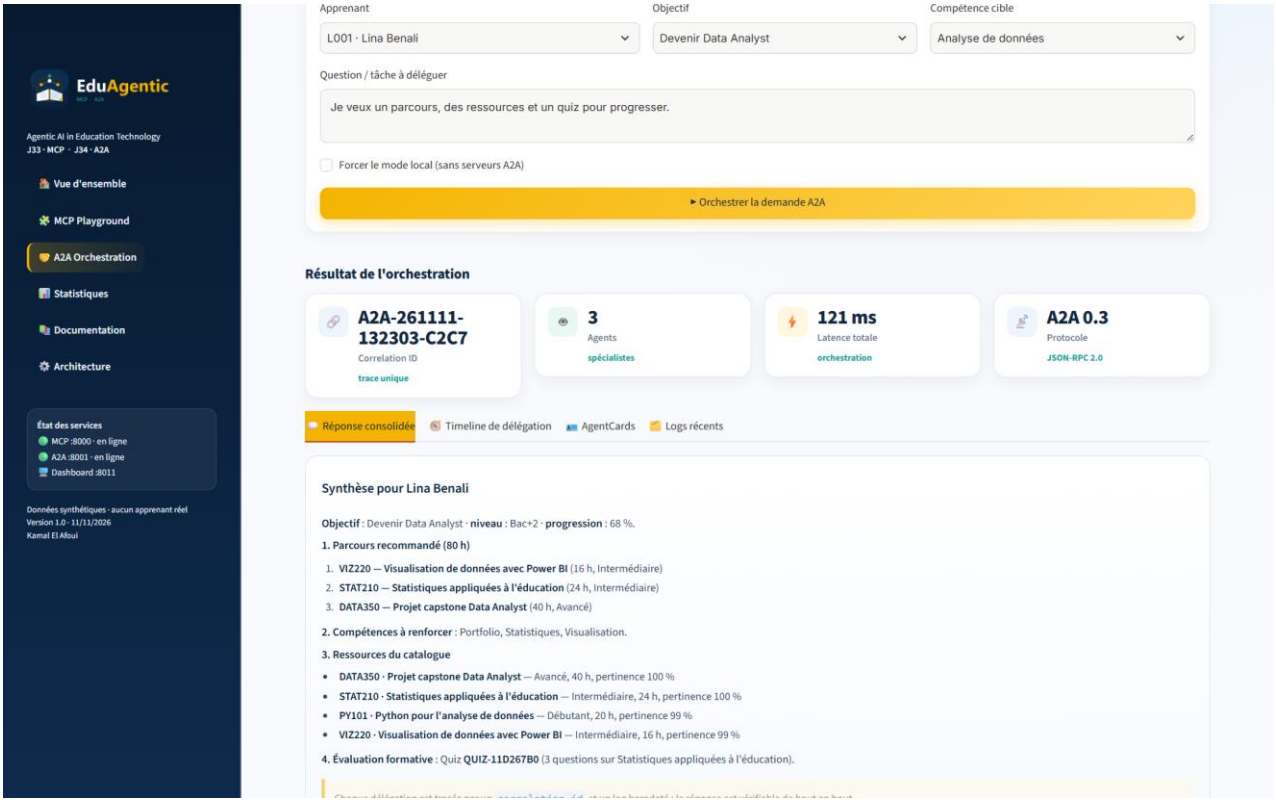


Figure 3 · Dashboard A2A : AgentCards en ligne, formulaire et lancement de l'orchestration.

8.1 Exemple de trace

Tableau 8 · Log de délégation : un même correlation_id relie tous les échanges.

Horodatage	Correlation ID	Échange inter-agents	Statut
09:15:01	A2A-7F3B21	Orchestrateur → Agent Orientation (diagnostic)	Envoyé
09:15:02	A2A-7F3B21	Orchestrateur → Agent Tuteur (ressources)	Envoyé
09:15:03	A2A-7F3B21	Orchestrateur → Agent Évaluation (quiz)	Envoyé
09:15:05	A2A-7F3B21	Orchestrateur → Agrégation (réponse finale)	Réponse envoyée

Lors du test HTTP du 11/11/2026, l'orchestration a duré 49.2 ms pour 3 agents, avec les statuts completed, completed, completed. Le correlation_id généré était A2A-261111-132825-2F42.

8.2 Ce que le correlation_id permet

- Retracer tout le parcours d'une demande, de l'orchestrateur jusqu'aux agents.
- Identifier rapidement l'agent qui a échoué ou ralenti.
- Comparer les performances entre agents et entre exécutions.
- Relier les logs aux artifacts et à la réponse finale.

8.3 Bénéfices pour l'architecture multi-agents

- **Spécialisation et découplage :** chaque agent reste concentré sur son domaine, sans connaître la logique interne des autres.
- **Scalabilité dynamique :** un nouvel agent rejoint l'écosystème en publiant simplement son AgentCard.
- **Audit et diagnostic :** le correlation_id et le log de délégation permettent d'isoler un échec et de mesurer la latence inter-agents.

9. Résultats et validation

Tableau 9 · Validation du système A2A.

Test	Résultat	Preuve
Découverte des cartes	3 / 3 agents en ligne	GET /discover
Délégation parallèle	3 / 3 completed	timeline du dashboard
Agrégation	Réponse unique + 3 artifacts	POST /orchestrate
Traçabilité	1 correlation_id, 3 lignes de log	data/a2a_live.jsonl
Dashboard	6 pages, 0 exception	tests Playwright

Interprétation

La latence de quelques dizaines de millisecondes montre que la découverte et la délégation locales sont très rapides. En production, il faudra ajouter l'authentification, le retry et un timeout par agent.

10. Sécurité et limites

Tableau 10 · Risques et mesures A2A.

Risque	Mesure	À renforcer
Agent non autorisé	Cartes locales, ports dédiés	Signature, registre, OAuth
Données personnelles	Données synthétiques	Minimisation, RGPD
Délégation en boucle	Orchestration bornée à 3 agents	Détection de cycles, budget
Panne d'un agent	Repli local et statut explicite	Retry, circuit breaker
Confiance dans la réponse	Sources et artifacts	Validation humaine

Le système est une implémentation pédagogique du profil A2A 0.3.0 : AgentCard, découverte, message/send, tasks/get et artifacts. Il ne prétend pas couvrir toutes les extensions de la spécification (streaming, push notifications, authentification avancée).

11. Mode d'emploi

```bash
# 1. Lancer les 3 agents + l'orchestrateur
python a2a/run_all.py

# 2. Vérifier la découverte
curl http://127.0.0.1:8001/discover

# 3. Dashboard
streamlit run dashboard/app.py --server.port 8011

# 4. Test direct d'orchestration
curl -X POST http://127.0.0.1:8001/orchestrate   -H "Content-Type: application/json"   -d
'{"learner_id":"L001","objective":"Devenir Data Analyst",
       "question":"Parcours, ressources et quiz"}'
```

Tableau 11 · Dépannage A2A.

Problème	Cause	Solution
Agent hors ligne	Processus non lancé	python a2a/run_all.py
Carte introuvable	Mauvais port	Vérifier 8101, 8102, 8103
Réponse sans quiz	Agent Évaluation indisponible	Vérifier /health du port 8102
Correlation_id absent	Appel direct sans metadata	Passer par /orchestrate

Conclusion

Le mini-projet J34 montre comment trois agents spécialisés peuvent collaborer sans se connaître : ils publient leurs compétences, l'orchestrateur les découvre et leur délègue des tâches. La réponse finale est plus riche qu'une réponse mono-agent, et surtout traçable grâce au correlation_id.

MCP et A2A ne sont pas concurrents : MCP outille un agent, A2A fait travailler plusieurs agents ensemble. La plateforme EduAgentic illustre cette complémentarité dans un contexte éducatif concret, avec des données synthétiques et une démonstration reproductible.

Checklist de livraison

☑ 3 AgentCards · ☑ Découverte HTTP · ☑ Délégation JSON-RPC · ☑ correlation_id · ☑ Workflow JSON · ☑ Dashboard · ☑ Documentation · ☑ Présentation · ☑ Vidéo.

Annexes

A. Glossaire

Tableau 12 · Glossaire A2A.

Terme	Définition
A2A	Agent-to-Agent : protocole de découverte et de délégation entre agents.
AgentCard	Carte d'identité d'un agent : compétences, endpoint, conditions.
Discovery	Découverte automatique des agents disponibles.
Delegation	Transmission d'une tâche à un agent spécialisé.
correlation_id	Identifiant unique qui relie tous les échanges d'une même demande.
Artifact	Résultat produit par un agent et retourné dans la Task.

B. Checklist de recette

- Les quatre AgentCards sont accessibles en JSON.
- L'orchestrateur découvre les trois agents en moins de 2 secondes.
- Les trois délégations utilisent le même correlation_id.
- Les trois artifacts sont présents dans la réponse.
- Le dashboard affiche la timeline et les logs récents.
- Le workflow JSON est importable et documenté.